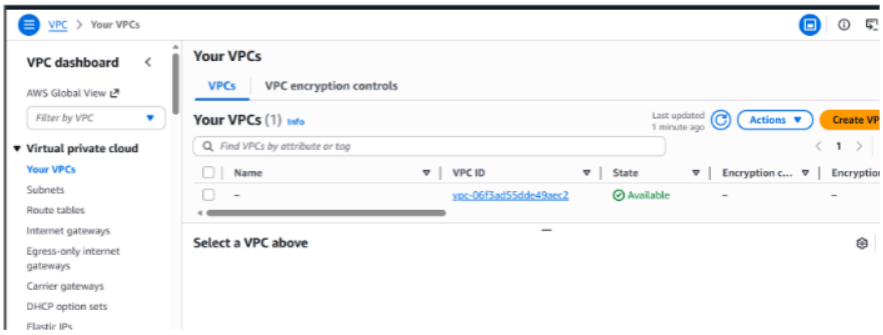
Autoscaling Task

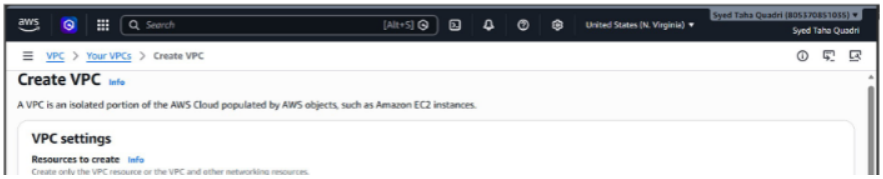
Task01: Create one VPC in N. Virginia region:

Let's create a VPC in N. Virginia region

Go to VPC Service



Click on create VPC

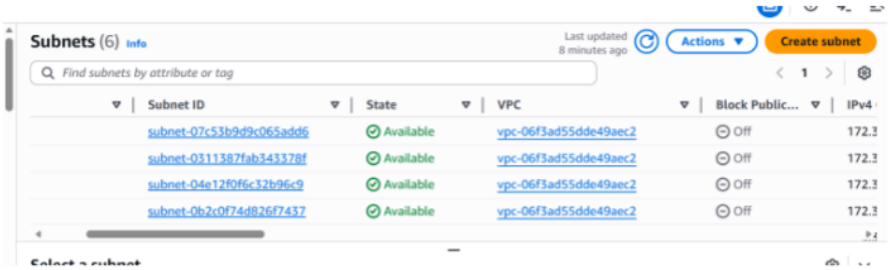


VPC is now created.

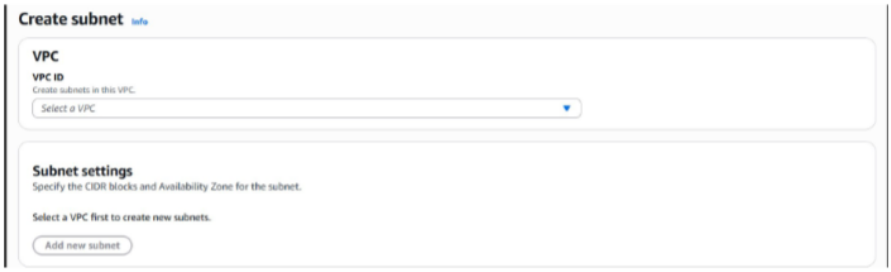
Task02: Create two subnets one public and one private subnet

Lets now create subnet

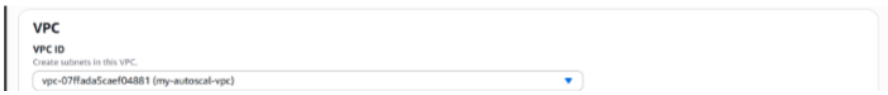
Click on subnets under VPC options

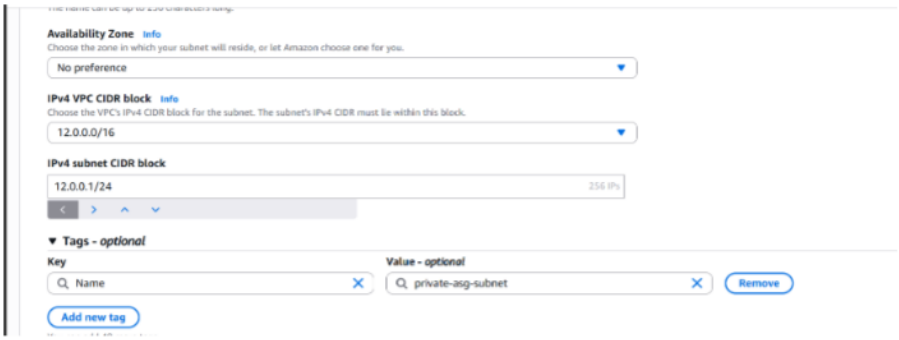


now click on create subnet

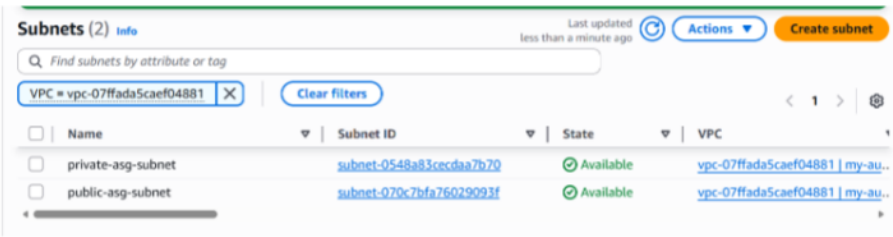


Selecting our newly created VPC



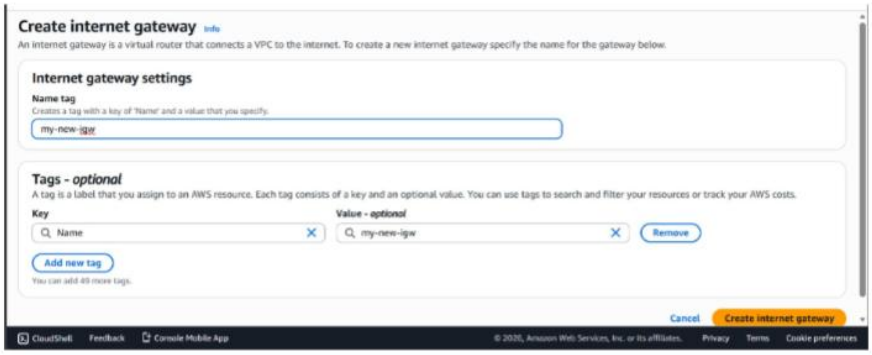


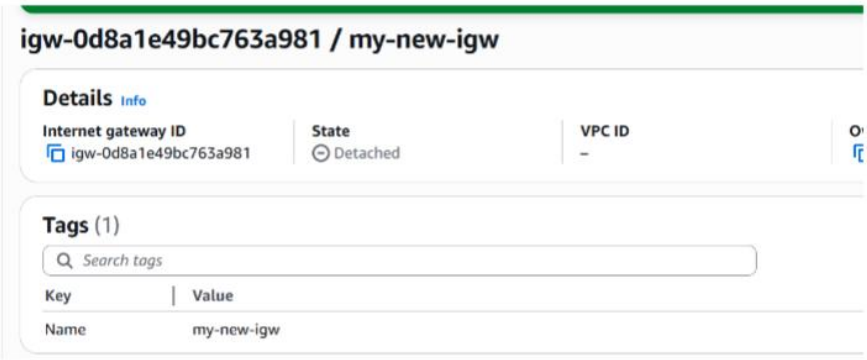
We have now created two subnets

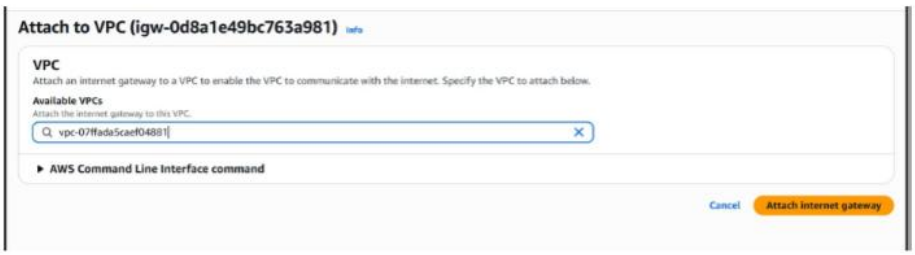


Task03 : Attach an IGW to vpc

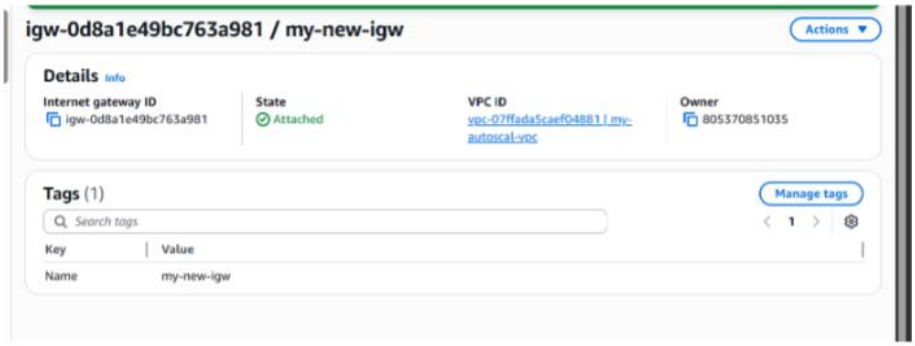
Now click on create Internet Gateway







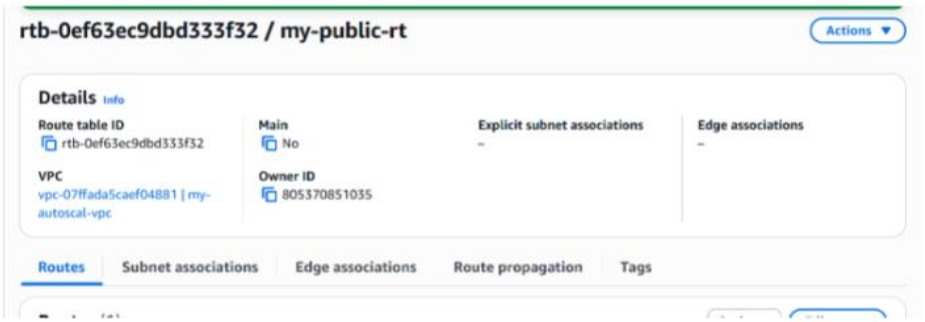
Now select our VPC



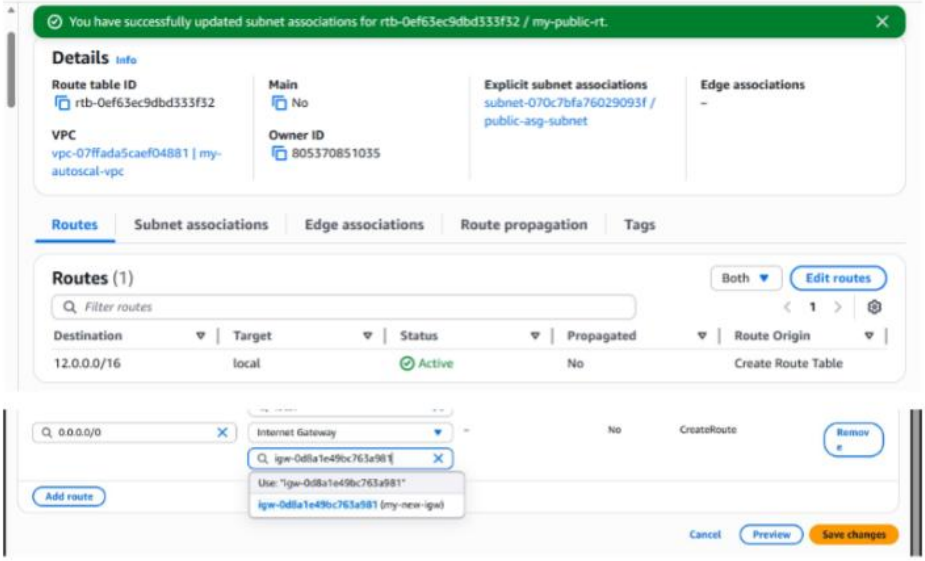
IGW is now attached to our VPC

Give route table a name and select VPC

Now create route table

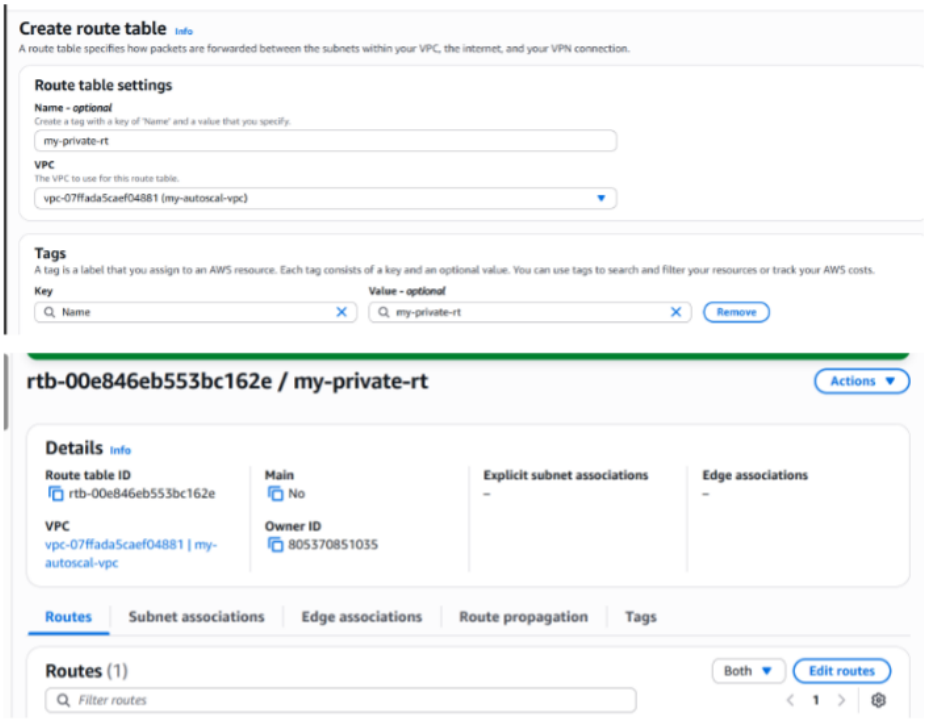


Associate this to a public subnet.

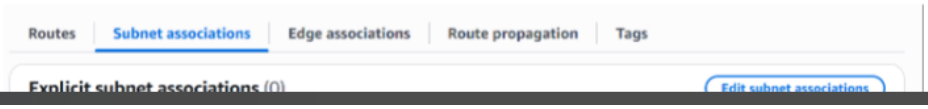


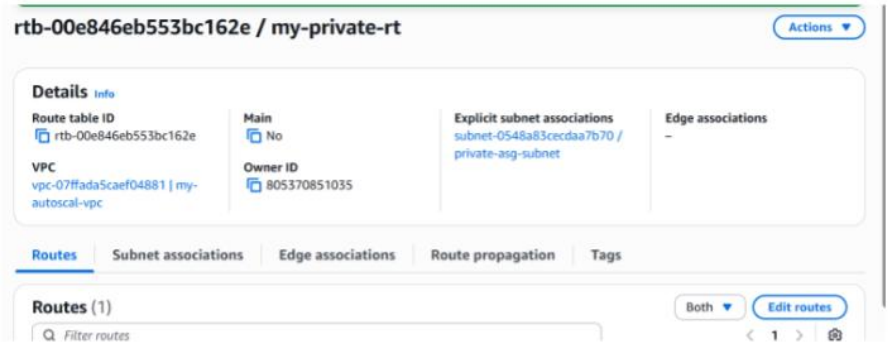
Attached internet gateway to public route table

We have attached our public route table to a public subnet



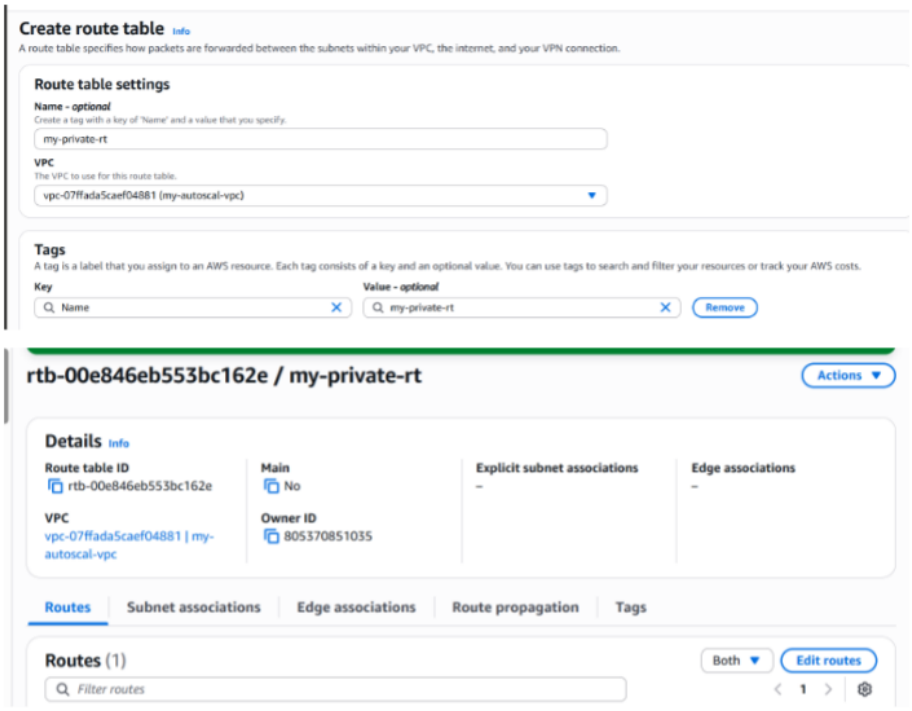
The private route table is now created let us now attach it to a private subnet by clicking on subnet associations



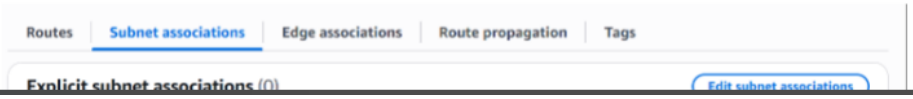


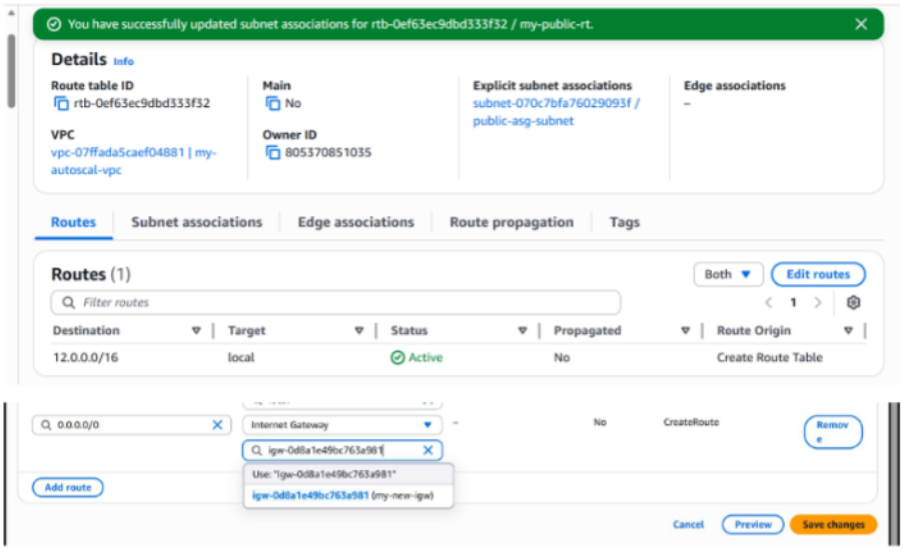
The route table is now attached to private subnet

Task05: Deploy a NAT gateway in public subnet and attach the nat gateway to private subnet.



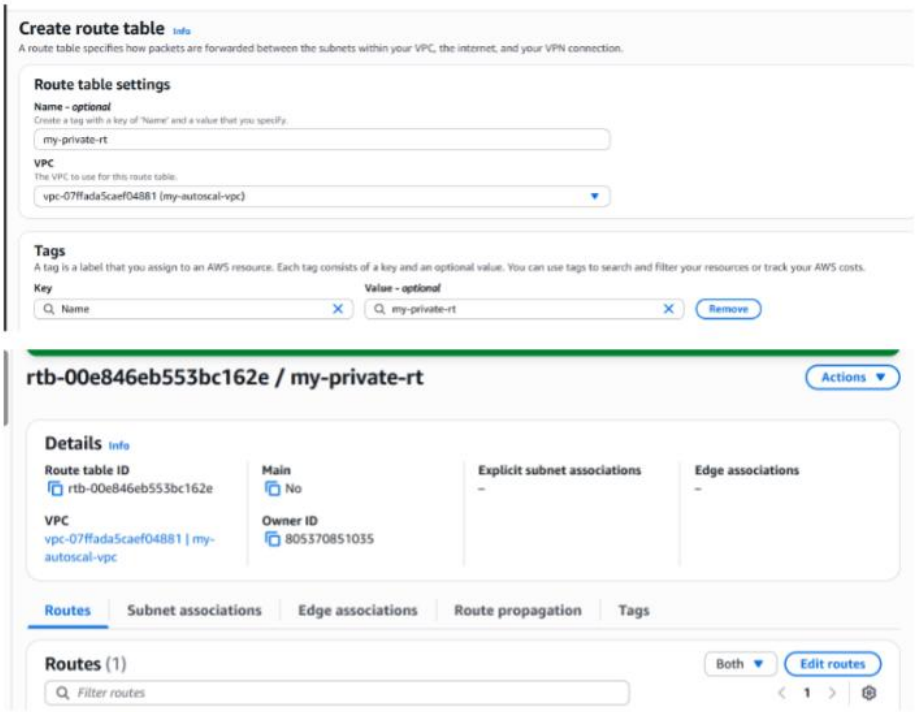
The private route table is now created let us now attach it to a private subnet by clicking on subnet associations



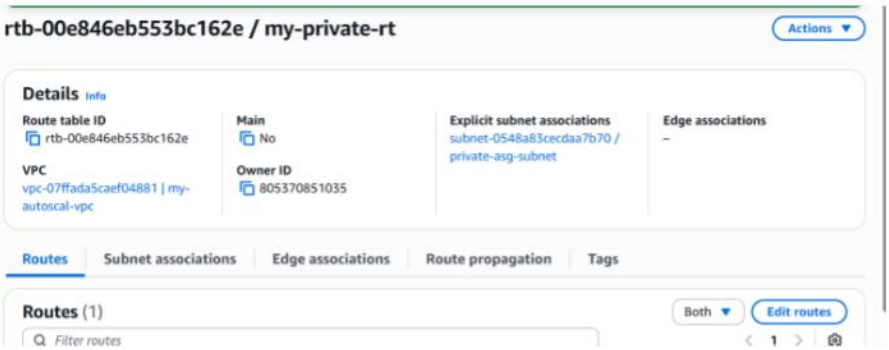


Attached internet gateway to public route table

We have attached our public route table to a public subnet



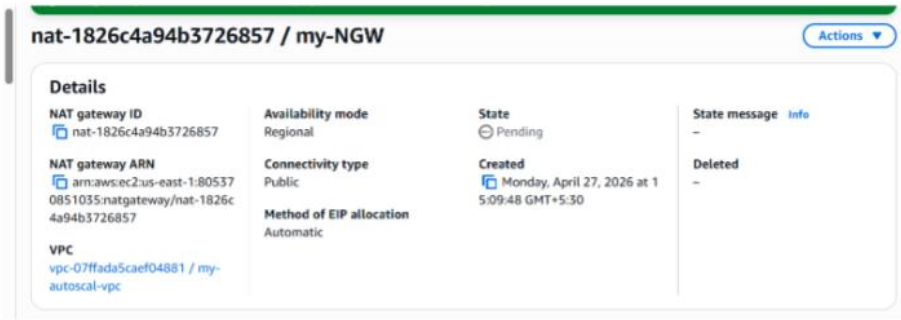
The private route table is now created let us now attach it to a private subnet by clicking on subnet associations



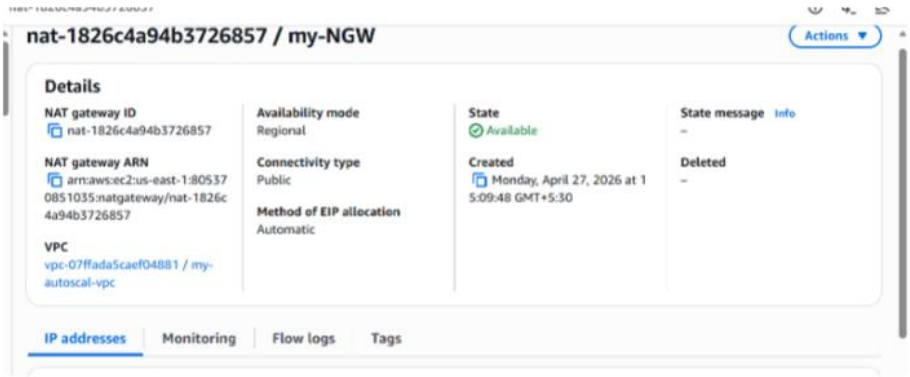
The route table is now attached to private subnet

Task05: Deploy a NAT gateway in public subnet and attach the nat gateway to private subnet.

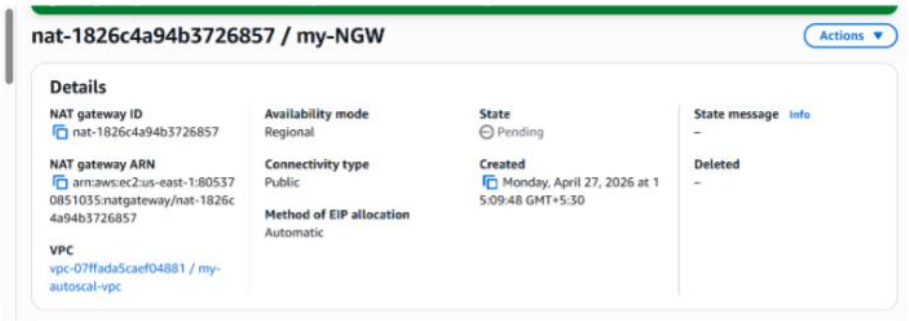
Give a name and attach it to vpc and then click on create



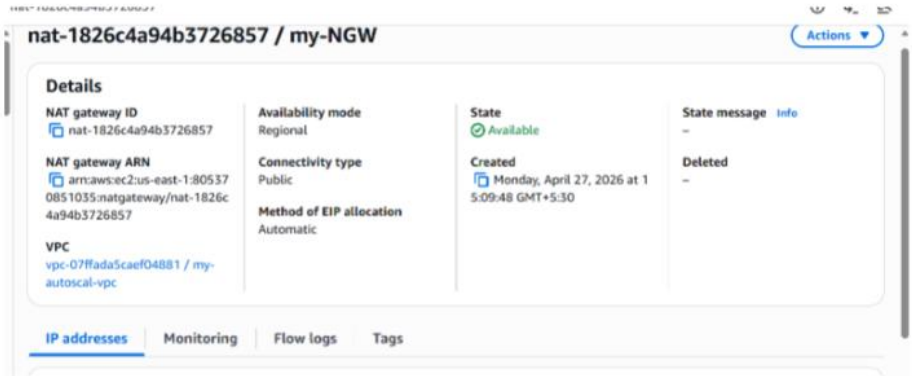
NAT gateway is created and is in pending state.



Give a name and attach it to vpc and then click on create

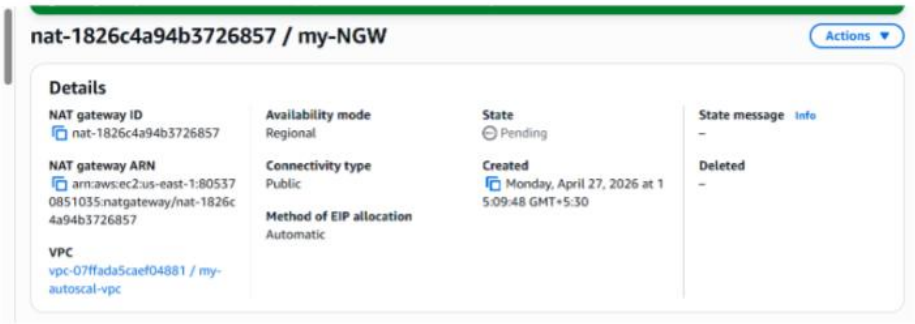


NAT gateway is created and is in pending state.

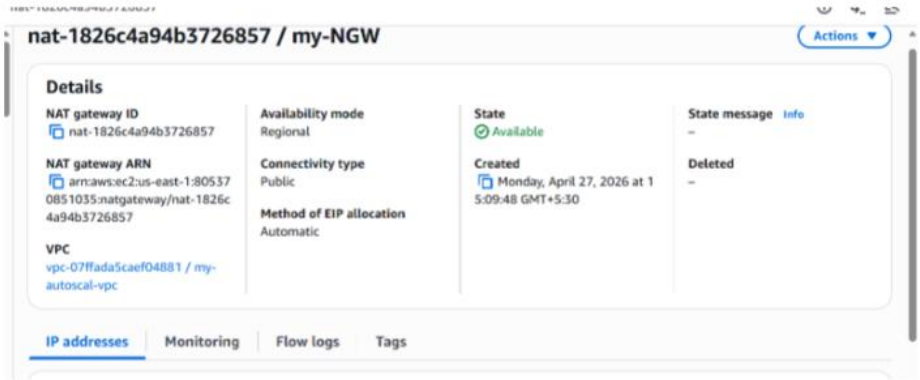


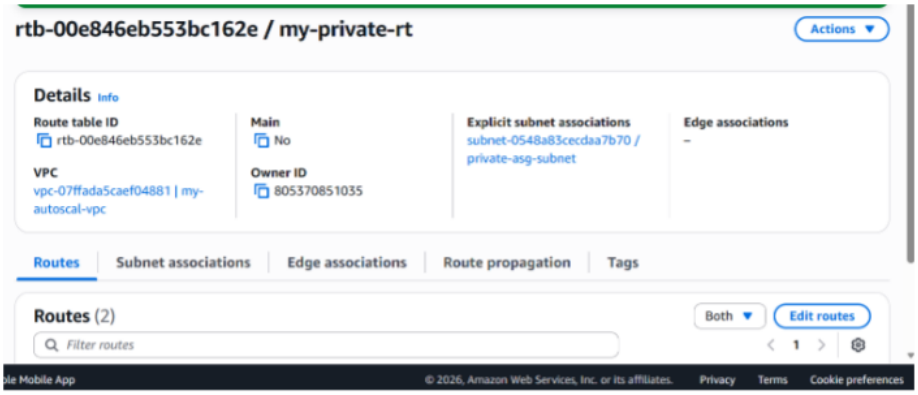
The NAT gateway is now in available state

Give a name and attach it to vpc and then click on create



NAT gateway is created and is in pending state.



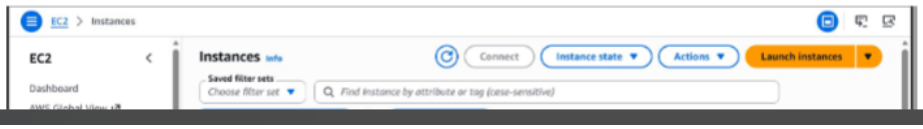


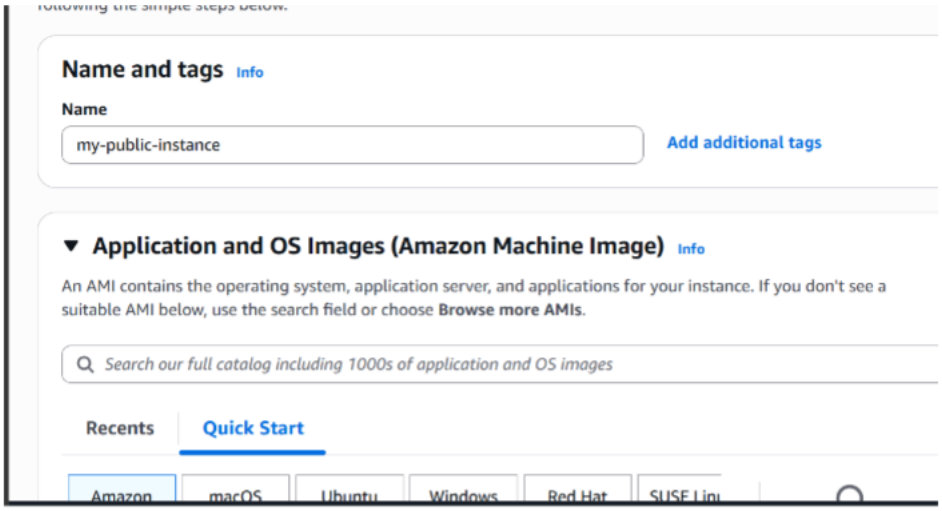
It is now attached to private subnet

Task06: Create one instance in public subnet and one in private subnet

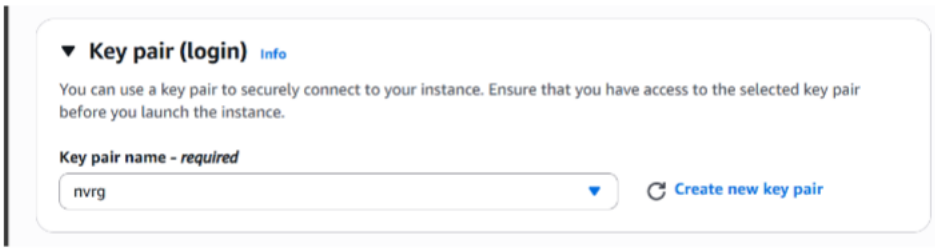
Lets now launch two instances one in public and one in private

Go to EC2 service





Give it a name and select key pair



Network settings Info

Edit

Network Info

vpc-06f3ad55dde49aec2

Subnet Info

No preference (Default subnet in any availability zone)

Auto-assign public IP Info

Enable

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

- (selected) Create security group
- Select existing security group

We'll create a new security group called '**launch-wizard-3**' with the following rules:

- [x] Allow SSH traffic from Anywhere

Network settings Info

VPC - *required* Info

vpc-07ffada5caef04881 (my-autoscal-vpc)
12.0.0.0/16

Subnet Info

subnet-070c7bfa76029093f public-asg-subnet
VPC: vpc-07ffada5caef04881 Owner: 805370851035
Availability Zone: us-east-1e (use1-az3) Zone type: Availability Zone
IP addresses available: 251 CIDR: 12.0.3.0/24)

Create new subnet

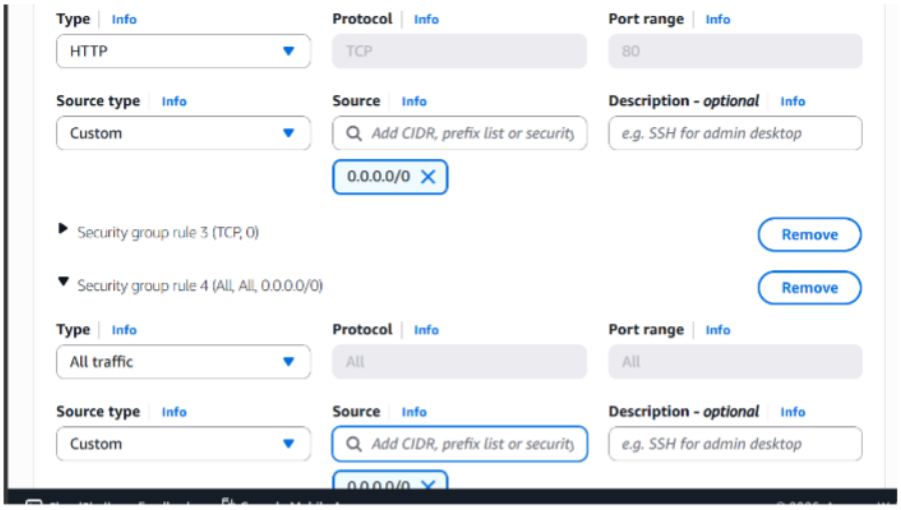
Auto-assign public IP Info

Enable

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

- (selected) Create security group
- Select existing security group



Allow HTTP and All traffic to check connectivity

Now click on launch instance

This is a public instance that we have created.

Name and tags Info

Name

my-private-instance

Add additional tags

▼ **Application and OS Images (Amazon Machine Image)** Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose **Browse more AMIs**.

Search our full catalog including 1000s of application and OS images

Recents | Quick Start

Number of instances | Info

1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.11....read more
ami-098e39bafa7e7303d

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Cancel | Launch instance

Preview code

Instance type

t3.micro — Free tier eligible
Family: t3 2 vCPU 1 GiB Memory Current generation: true
On-Demand Linux base pricing: 0.0104 USD per Hour
On-Demand Windows base pricing: 0.0196 USD per Hour
On-Demand Ubuntu Pro base pricing: 0.0139 USD per Hour
On-Demand SUSE base pricing: 0.0104 USD per Hour
On-Demand RHEL base pricing: 0.0392 USD per Hour

All generations

Compare instance types

Additional costs apply for AMIs with pre-installed software

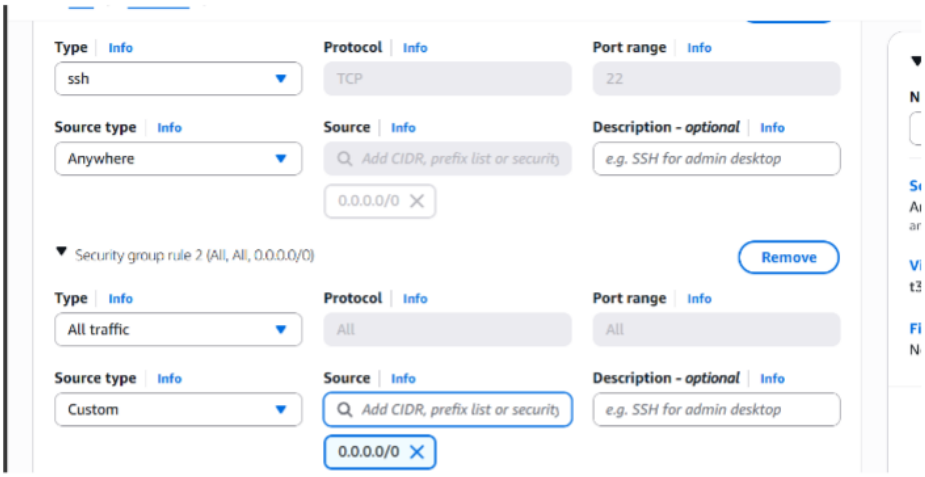
▼ **Key pair (login)** Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

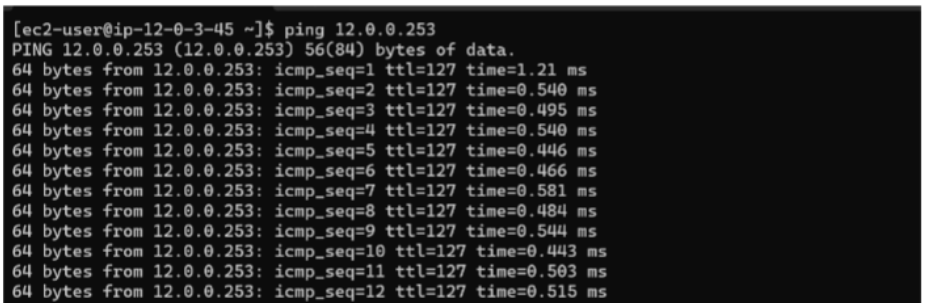
Key pair name - *required*

nvrg

Create new key pair



We have ssh to our public instance and we are able to ping to our instance in private subnet



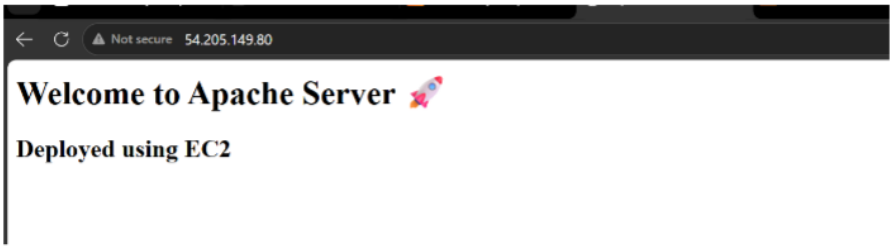
```
[ec2-user@ip-12-0-3-45 ~]$ ping 12.0.0.253
PING 12.0.0.253 (12.0.0.253) 56(84) bytes of data.
64 bytes from 12.0.0.253: icmp_seq=1 ttl=127 time=1.21 ms
64 bytes from 12.0.0.253: icmp_seq=2 ttl=127 time=0.540 ms
64 bytes from 12.0.0.253: icmp_seq=3 ttl=127 time=0.495 ms
64 bytes from 12.0.0.253: icmp_seq=4 ttl=127 time=0.540 ms
64 bytes from 12.0.0.253: icmp_seq=5 ttl=127 time=0.446 ms
64 bytes from 12.0.0.253: icmp_seq=6 ttl=127 time=0.466 ms
64 bytes from 12.0.0.253: icmp_seq=7 ttl=127 time=0.581 ms
64 bytes from 12.0.0.253: icmp_seq=8 ttl=127 time=0.484 ms
64 bytes from 12.0.0.253: icmp_seq=9 ttl=127 time=0.544 ms
64 bytes from 12.0.0.253: icmp_seq=10 ttl=127 time=0.443 ms
64 bytes from 12.0.0.253: icmp_seq=11 ttl=127 time=0.503 ms
64 bytes from 12.0.0.253: icmp_seq=12 ttl=127 time=0.515 ms
```

```
[ec2-user@ip-12-0-3-45 ~]$ sudo yum install httpd -y
Last metadata expiration check: 0:01:51 ago on Mon Apr 27 10:34:54 2026.
Dependencies resolved.
================================================================================================================================
 Package                          Architecture        Version                          Repository                  Size
================================================================================================================================
Installing:
 httpd                            x86_64              2.4.66-1.amzn2023.0.1            amazonlinux                 47 k
Installing dependencies:
 apr                              x86_64              1.7.5-1.amzn2023.0.4             amazonlinux                129 k
 apr-util                         x86_64              1.6.3-1.amzn2023.0.2             amazonlinux                 97 k
 apr-util-lmdb                    x86_64              1.6.3-1.amzn2023.0.2             amazonlinux                 13 k
 generic-logos-httpd              noarch              18.0.0-12.amzn2023.0.3           amazonlinux                 19 k
 httpd-core                       x86_64              2.4.66-1.amzn2023.0.1            amazonlinux                1.4 M
 httpd-filesystem                 noarch              2.4.66-1.amzn2023.0.1            amazonlinux                 13 k
 httpd-tools                      x86_64              2.4.66-1.amzn2023.0.1            amazonlinux                 81 k
 libbrotli                        x86_64              1.0.9-4.amzn2023.0.2             amazonlinux                315 k
 mailcap                          noarch              2.1.49-3.amzn2023.0.3            amazonlinux                 33 k
Installing weak dependencies:
 apr-util-openssl                 x86_64              1.6.3-1.amzn2023.0.2             amazonlinux                 15 k
 mod_http2                        x86_64              2.0.27-1.amzn2023.0.3            amazonlinux                166 k
 mod_lua                          x86_64              2.4.66-1.amzn2023.0.1            amazonlinux                 60 k

Transaction Summary
================================================================================================================================
Install  13 Packages

Total download size: 2.4 M
```

```
Installed:
  apr-1.7.5-1.amzn2023.0.4.x86_64                                   apr-util-1.6.3-1.amzn2023.0.2.x86_64
  apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64                         apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch                 httpd-2.4.66-1.amzn2023.0.1.x86_64
  httpd-core-2.4.66-1.amzn2023.0.1.x86_64                           httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch
  httpd-tools-2.4.66-1.amzn2023.0.1.x86_64                          libbrotli-1.0.9-4.amzn2023.0.2.x86_64
  mailcap-2.1.49-3.amzn2023.0.3.noarch                              mod_http2-2.0.27-1.amzn2023.0.3.x86_64
  mod_lua-2.4.66-1.amzn2023.0.1.x86_64
```

Go to /var/www/html

```
  GNU nano 8.3                                                    index.html
<html>
  <head>
    <title>My First AWS Web Server</title>
  </head>
  <body>
    <h1>Welcome to Apache Server 🚀</h1>
    <h2>Deployed using EC2</h2>
  </body>
</html>
```



We are now able to access our website

Let us now deploy in private instance

```
[ec2-user@ip-12-0-3-45 ~]$ ssh -i nvrg.pem 12.0.0.253
       #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-12-0-0-253 ~]$
```

We are into our private instance now

```
[ec2-user@ip-12-0-0-253 ~]$ sudo yum install httpd
Last metadata expiration check: 0:00:11 ago on Mon Apr 27 10:45:48 2026.
Dependencies resolved.
================================================================================
 Package                    Architecture        Version
================================================================================
Installing:
 httpd                      x86_64              2.4.66-1.amzn2023.0.1
Installing dependencies:
 apr                        x86_64              1.7.5-1.amzn2023.0.4
 apr-util                   x86_64              1.6.3-1.amzn2023.0.2
 apr-util-lmdb              x86_64              1.6.3-1.amzn2023.0.2
 generic-logos-httpd        noarch              18.0.0-12.amzn2023.0.3
 httpd-core                 x86_64              2.4.66-1.amzn2023.0.1
 httpd-filesystem           noarch              2.4.66-1.amzn2023.0.1
 httpd-tools                x86_64              2.4.66-1.amzn2023.0.1
 libbrotli                  x86_64              1.0.9-4.amzn2023.0.2
 mailcap                    noarch              2.1.49-3.amzn2023.0.3
Installing weak dependencies:
 apr-util-openssl           x86_64              1.6.3-1.amzn2023.0.2
 mod_http2                  x86_64              2.0.27-1.amzn2023.0.3
 mod_lua                    x86_64              2.4.66-1.amzn2023.0.1
```

```
Installed:
  apr-1.7.5-1.amzn2023.0.4.x86_64                          apr-util-1.6.3-1.amzn2023.0.2.x86_64
  apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64                apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch        httpd-2.4.66-1.amzn2023.0.1.x86_64
  httpd-core-2.4.66-1.amzn2023.0.1.x86_64                  httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch
  httpd-tools-2.4.66-1.amzn2023.0.1.x86_64                 libbrotli-1.0.9-4.amzn2023.0.2.x86_64
  mailcap-2.1.49-3.amzn2023.0.3.noarch                     mod_http2-2.0.27-1.amzn2023.0.3.x86_64
  mod_lua-2.4.66-1.amzn2023.0.1.x86_64
```

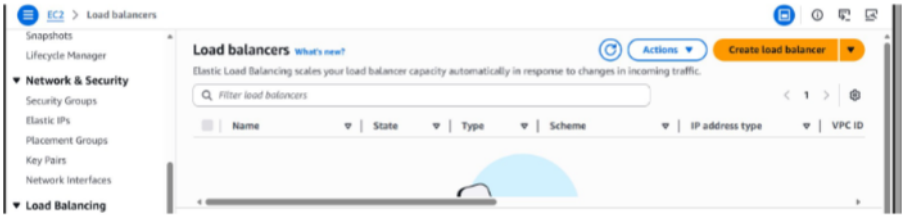
```
[ec2-user@ip-12-0-0-253 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
     Active: active (running) since Mon 2026-04-27 10:46:34 UTC; 16s ago
       Docs: man:httpd.service(8)
   Main PID: 25994 (httpd)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec
      Tasks: 177 (limit: 1014)
     Memory: 13.5M
        CPU: 77ms
     CGroup: /system.slice/httpd.service
             ├─25994 /usr/sbin/httpd -DFOREGROUND
             ├─25995 /usr/sbin/httpd -DFOREGROUND
             ├─25996 /usr/sbin/httpd -DFOREGROUND
             ├─25997 /usr/sbin/httpd -DFOREGROUND
             └─26004 /usr/sbin/httpd -DFOREGROUND
```

```
[ec2-user@ip-12-0-0-253 html]$ curl 12.0.0.253
<html>
  <head>
    <title>My First AWS Web Server</title>
  </head>
  <body>
    <h1>Welcome to Apache Server 🚀</h1>
    <h2>Deployed using EC2</h2>
  </body>
</html>
[ec2-user@ip-12-0-0-253 html]$
```

Our static website is now working in private instance as well.

Task08: Create one application load balancer and attach it to both EC2 instances.

Lets go to ec2 and select load balancer



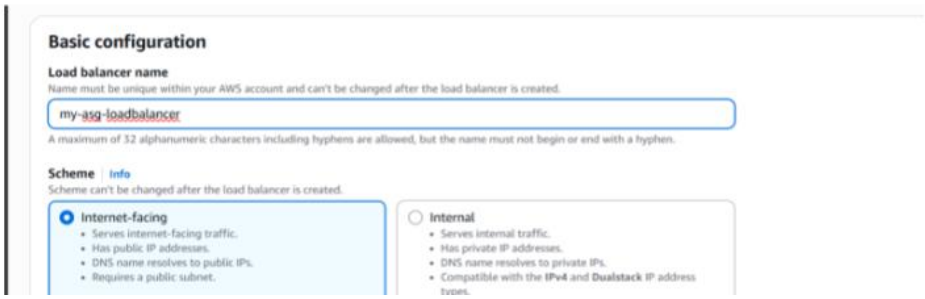
Click on create load balancer

Select application load balancer

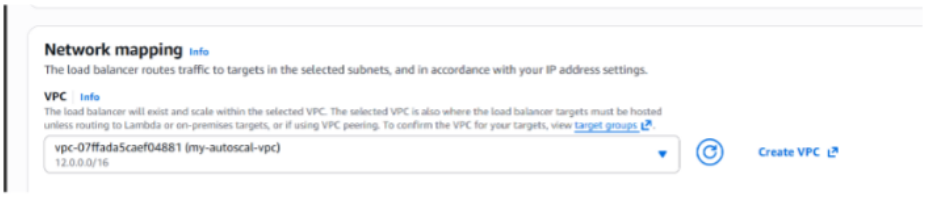
Choose an Application Load Balancer when you need a flexible feature set for your applications with HTTP and HTTPS traffic. Operating at the request level, Application Load Balancers provide advanced routing and visibility features targeted at application architectures, including microservices and containers.

Create

Give it a name and select internet facing



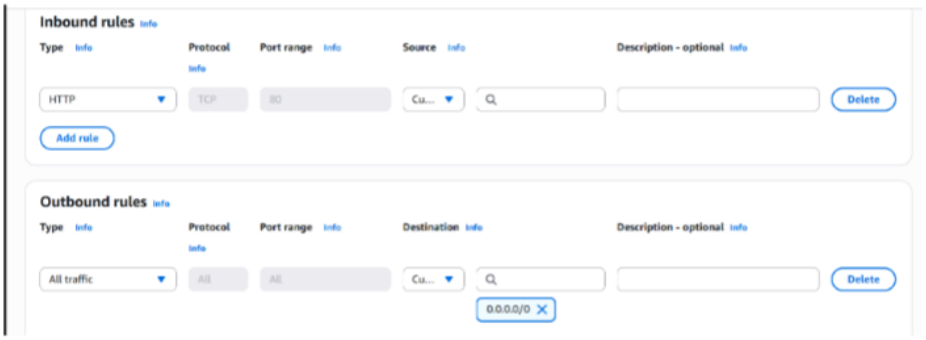
Select VPC



Now let us create a security group and give

Inbound 80

Outbound all traffic access.



Now create a Target Group

Target group name
Name must be unique per Region per AWS account.

test-target-group

Accepts: a-z, A-Z, 0-9, and hyphen (-). Can't begin or end with hyphen. 1-32 total characters; Count: 17/32

Protocol
Protocol for communication between the load balancer and targets.

HTTP

Port
Port number where targets receive traffic. Can be overridden for individual targets during registration.

80

1-65535

IP address type
Only targets with the indicated IP address type can be registered to this target group.

- IPv4
  Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.
- IPv6
  Each instance you register must have an assigned primary IPv6 address. This is configured on the instance's default network interface (eth0). Learn more

VPC
Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.

vpc-07ffada5caef04881 (my-autoscal-vpc)
12.0.0.0/16

Create VPC

Protocol version

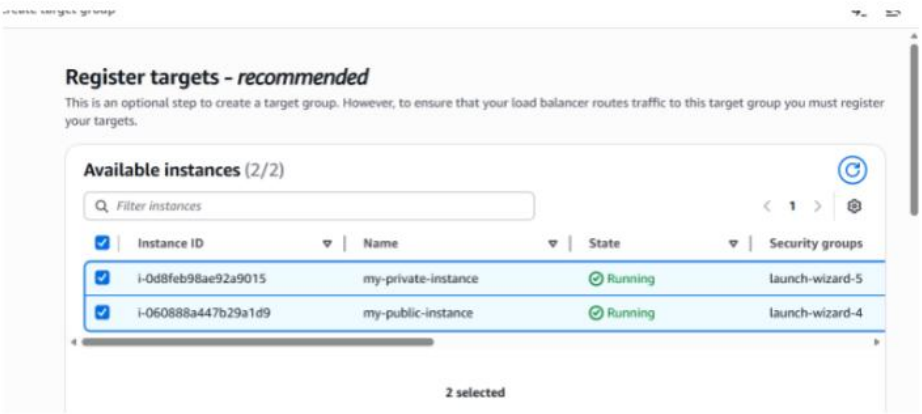
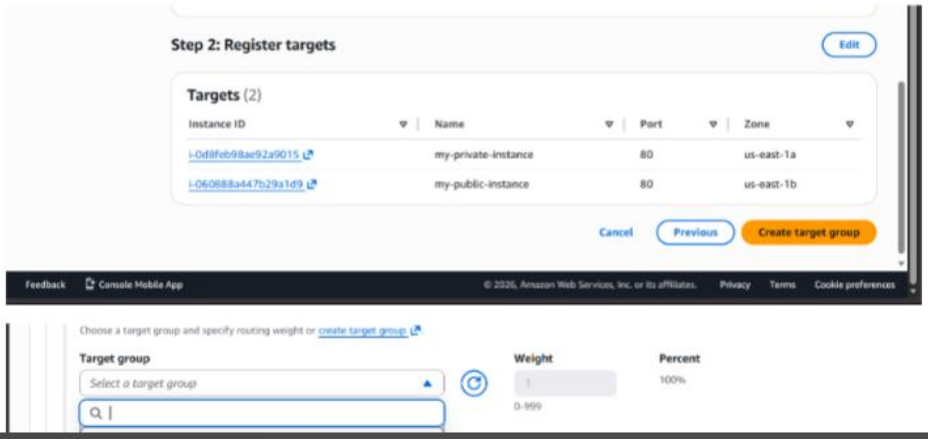
- HTTP1
  Send requests to targets using HTTP/1.1. Supported when the request protocol is HTTP/1.1 or HTTP/2.
- HTTP2
  Send requests to targets using HTTP/2. Supported when the request protocol is HTTP/2 or gRPC.

Health checks
The associated load balancer periodically sends requests, per the settings below, to the registered targets to test their status.

Health check protocol

HTTP

Health check path
Use the default path of "/" to perform health checks on the root, or specify a custom path if preferred.



Click on create target group now